SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	P H Devesh
Register Number	1911260236
Course Code & Name	CSA0209 & C Programming
Slot	B
Assignment / Task Title	C02 · Logic Analysis and Design Challenges
GitHub Repository URL	https://github.com/DeveshPH/C-CSA0209
Submission Date	_____

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

CSA0209 — C PROGRAMMING

C02 – Logic Analysis and Design Challenges

Assignment overview: The answers below follow the seven questions and evidence requirements visible in the supplied SIMATS Online Programming Lab screenshots. Code is written in standard C.

Q1 – Precedence, Associativity and the Discount Bug (6 Marks)

Problem and assumptions

The pharmacy must calculate gross amount, apply a 15% discount only for senior citizens, calculate 5% GST on the discounted taxable amount, and subtract Rs. 50 only when the amount after GST is strictly greater than Rs. 1,000. For the three worked bills below, price and quantity are integers, while the corrected calculation uses double to preserve paise.

Operator analysis of the faulty statements

In `price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100`, multiplication and division have equal precedence and associate left-to-right. The discount is computed from gross amount, but GST is also computed from gross amount rather than from the discounted taxable amount. With integer operands, each division truncates its fractional part. The ternary operator applies the Rs. 50 reduction only when `total > 1000`; exactly Rs. 1,000 is not reduced.

The expression `count = count++ + 1;` has undefined behaviour in C because the modification of `count` by the post-increment and the assignment are unsequenced relative to one another. Use `count++;` or `count += 1;` instead.

Three predictions using the faulty integer expression

Bill	Inputs (price × qty)	Senior?	Faulty total	Faulty net
1	100 × 2 = 200	Yes	200 − 30 + 10 = 180	180
2	500 × 3 = 1,500	Yes	1,500 − 225 + 75 = 1,350	1,300
3	250 × 4 = 1,000	No	1,000 + 50 = 1,050	1,000

Correct calculation and expected values

Bill 1: gross Rs. 200; discount Rs. 30; taxable Rs. 170; GST Rs. 8.50; total/net Rs. 178.50. Bill 2: gross Rs. 1,500; discount Rs. 225; taxable Rs. 1,275; GST Rs. 63.75; total Rs. 1,338.75; final net Rs. 1,288.75. Bill 3: gross Rs. 1,000; no discount; GST Rs. 50; total Rs. 1,050; final net Rs. 1,000.

Corrected C program

```
#include <stdio.h>

int main(void)
{
    double price, qty, gross, discount, taxable, gst, total, net;
    int senior, count = 0;

    /* Demonstration bill: Rs. 500 x 3, senior citizen */
    price = 500.0;
    qty = 3.0;
    senior = 1;

    gross = price * qty;
    discount = senior ? gross * 15.0 / 100.0 : 0.0;
    taxable = gross - discount;
    gst = taxable * 5.0 / 100.0;
    total = taxable + gst;
    net = (total > 1000.0) ? total - 50.0 : total;

    count++;  /* well-defined increment */

    printf("Gross: Rs. %.2f\\n", gross);
    printf("Discount: Rs. %.2f\\n", discount);
    printf("Taxable: Rs. %.2f\\n", taxable);
    printf("GST: Rs. %.2f\\n", gst);
    printf("Total: Rs. %.2f\\n", total);
    printf("Net payable: Rs. %.2f\\n", net);
```

```
    printf("Bills counted: %d\\n", count);
    return 0;
}
```

Sample output

```
Gross: Rs. 1500.00
Discount: Rs. 225.00
Taxable: Rs. 1275.00
GST: Rs. 63.75
Total: Rs. 1338.75
Net payable: Rs. 1288.75
Bills counted: 1
```

Design decision and complexity

Intermediate variables are preferred because each business step is visible and independently testable. The number of arithmetic operations remains constant, so time is O(1). A few scalar temporary variables use O(1) extra space. The tiny space cost is justified by improved correctness, maintainability and auditability.

Q2 – When Does Short-Circuit Evaluation Matter? (4 Marks)

A concession is granted when (senior citizen AND eligible class) OR verified disability card. Server verification is the expensive operation.

Short-circuit rules

A `&&` B does not evaluate B when A is false. A `||` B does not evaluate B when A is true. Bitwise & and | evaluate both operands; they are not substitutes for logical short-circuit operators. The conditions should be ordered so inexpensive, likely-false checks occur before an expensive operation, provided no audit requirement demands the call.

Reproducible experiment (1,000 passengers)

For a transparent count, assume 1,000 records: 200 passengers satisfy both senior and eligible-class checks, 100 are senior but in an ineligible class, and 700 are not senior. The server verification function is called only when reached. The following two arrangements are compared.

```
int calls = 0;

int verify_card(int passenger_id)
{
    calls++;
    /* In a real system, contact the verification server here. */
    return card_is_valid(passenger_id);
}

/* Cheap checks first: server call only if concession is not
   already granted by senior + eligible class. */
grant = (senior && eligible) || verify_card(id);

/* Expensive check first: it is called for every passenger
   because it is the first operand. */
grant = verify_card(id) || (senior && eligible);
```

Condition order	Server calls for 1,000	Reason
(senior && eligible) || verify_card()	800	200 qualify in the first group and short-circuit; the other 800 reach verification.
verify_card() || (senior && eligible)	1,000	Verification is the first operand, so it is always called.

Note: The call count above follows the stated 200-person first group. If the data distribution changes, the exact count changes; the principle does not.

Changed condition: mandatory audit logging

If the server function must record every passenger it checks, then every passenger must be sent to it. In that changed requirement, place the verification call in a separate statement before deciding the concession. Short-circuiting must not skip a required audit event. The number of calls is then exactly n for n passengers.

```
int card_ok = verify_card(id);   /* mandatory audit for everyone */
grant = (senior && eligible) || card_ok;
```

Nested if statements can express the same short-circuit decision when the server call is only conditionally required. Using & or | may force both operands to be evaluated and can cause unnecessary server calls. With short-circuiting, the worst-case number of calls is O(n), and the extra counter/temporary storage is O(1).

Q3 – Engineer a Device-Status Register with Bitwise Operators (6 Marks)

One unsigned byte stores the states of eight devices. Device number dev uses bit dev, where dev ranges from 0 to 7. The supplied enum assigns LIGHT=0, FAN=1, AC=2, HEATER=3 and PUMP=4.

Correct bit-manipulation functions

```
#include <stdio.h>

enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    return (unsigned char)(reg | (1u << dev));
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return (unsigned char)(reg & ~(1u << dev));
}

unsigned char toggle(unsigned char reg, int dev)
{
    return (unsigned char)(reg ^ (1u << dev));
}

int is_on(unsigned char reg, int dev)
{
    return (reg & (1u << dev)) != 0;
}

void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--)
        putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\\n');
}

int count_on(unsigned char reg)
{
    int count = 0;
    for (int bit = 0; bit < 8; bit++)
        if ((reg >> bit) & 1u)
            count++;
    return count;
}

int main(void)
{
    unsigned char reg = 0;

    reg = switch_on(reg, LIGHT); /* 00000001 */
    reg = switch_on(reg, AC);    /* 00000101 */
    show(reg);

    reg = toggle(reg, FAN);      /* 00000111 */
    reg = switch_off(reg, LIGHT);/* 00000110 */
    show(reg);

    printf("AC is %s, PUMP is %s\\n",
           is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    printf("Devices ON: %d\\n", count_on(reg));
    return 0;
}
```

Predicted register values and output

Statement	Register (binary)	Hex
Initial reg = 0	00000000	0x00
Switch LIGHT on	00000001	0x01
Switch AC on	00000101	0x05
Toggle FAN	00000111	0x07
Switch LIGHT off	00000110	0x06

```
00000101
00000110
AC is ON, PUMP is OFF
Devices ON: 2
```

Precedence and signedness

Shift has higher precedence than bitwise AND. Therefore `reg & 1u << dev` is parsed as `reg & (1u << dev)`. The explicit form `(reg & 1u) << dev` first extracts only bit 0 and then shifts it; it does not test bit dev in the original register. Use unsigned values so left shifts are well-defined for the intended bit positions and avoid signed-char sign extension. The mask must be parenthesized for clarity.

Space and time comparison

A packed register uses 1 byte per home for eight states; an array of eight char values uses 8 bytes. For 1,000,000 homes, that is approximately 1,000,000 bytes versus 8,000,000 bytes, saving about 7 MB (decimal). Set, clear, toggle and test are O(1) in either representation. Counting ON devices in the byte takes at most eight checks, hence O(1) for a fixed eight-device system; counting an array of eight entries also takes O(8)=O(1), but uses more storage.

Q4 – else-if Ladder or switch? (6 Marks)

Grade rules from the supplied question: 90–100 S, 80–89 A, 70–79 B, 60–69 C, 50–59 D, below 50 F; marks outside 0–100 are invalid.

Implementation using else-if

```
char grade_if(int marks)
{
    if (marks < 0 || marks > 100) return '?';
    if (marks >= 90) return 'S';
    else if (marks >= 80) return 'A';
    else if (marks >= 70) return 'B';
    else if (marks >= 60) return 'C';
    else if (marks >= 50) return 'D';
    else return 'F';
}
```

Implementation using switch on marks / 10

```
char grade_switch(int marks)
{
    if (marks < 0 || marks > 100) return '?';

    switch (marks / 10) {
        case 10:
        case 9:  return 'S';
        case 8:  return 'A';
        case 7:  return 'B';
        case 6:  return 'C';
        case 5:  return 'D';
        default: return 'F'; /* 0 through 4 */
    }
}
```

Boundary tests

Marks	Expected grade	Reason
100	S	Top valid boundary
90	S	Lower S boundary
89	A	Upper A boundary
80	A	Lower A boundary
79	B	Upper B boundary
50	D	Lower D boundary
49	F	Below D range
0	F	Lowest valid mark
-1 / 101	?	Invalid range

Both versions handle all listed inputs correctly when the range guard is present. Without the guard, negative marks can be misclassified because C integer division truncates toward zero; marks above 100 may be treated as S by the switch version. A missing break in a switch causes fall-through into subsequent cases; return statements avoid this problem. switch cannot directly test a range such as 75–84 because case labels must be integer constant expressions, not relational ranges.

For changing grade boundaries, an else-if ladder is generally clearer for ranges. A lookup table is compact and O(1) per grade, but it requires a table with 11 entries (indices 0–10) and careful handling of invalid marks. Scholarship classification depends on both grade and family-income bands, but the screenshots do not specify the income thresholds or scholarship outcomes. Therefore, those rules must be supplied before exact scholarship code can be written; a nested if or lookup table can then represent the two-dimensional rule.

Test both implementations for every boundary and one mark immediately below and above each boundary. For example: -1, 0, 49, 50, 59, 60, 69, 70, 79, 80, 89, 90, 100, 101.

Q5 – Count the Work Done by a Loop (7 Marks)

A number is prime if it is at least 2 and has no divisor other than 1 and itself. If n has a divisor greater than its square root, it must also have a paired divisor smaller than the square root. Therefore, it is sufficient to test divisors while i ≤ n/i.

Correct improved implementation with division count

```
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n)
{
    unsigned long i;
    checks = 0;

    if (n < 2) return 0;

    for (i = 2; i <= n / i; i++) {
        checks++;
        if (n % i == 0)
            return 0;
    }
    return 1;
}

int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};
    int k;

    for (k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%lu %s divisions = %lu\\n",
               tickets[k], p ? "prime" : "not prime", checks);
    }
    return 0;
}
```

Expected output

```
97 prime divisions = 8
100 not prime divisions = 1
7919 prime divisions = 87
1000003 prime divisions = 999
```

Why this loop condition is safer

The condition `i <= n / i` avoids calculating `i * i`, which could overflow for a sufficiently large unsigned value. For n=2 or n=3, the loop body is not entered and the function correctly returns prime. A do-while loop executes its body at least once, so it needs a special small-number check before entering the loop.

Equivalent while and do-while versions

```
/* while version: same safe condition */
i = 2;
while (i <= n / i) {
    checks++;
    if (n % i == 0) return 0;
    i++;
}

/* do-while version: guard n < 2 first, and ensure that
   n = 2 or n = 3 does not test a divisor */
i = 2;
if (n < 2) return 0;
do {
    if (i > n / i) break;
    checks++;
    if (n % i == 0) return 0;
    i++;
} while (i <= n / i);
return 1;
```

If the condition is changed to i < n, a prime number tests all divisors from 2 to n−1. For 1,000,003, that is 1,000,001 divisions instead of 999. Trial division takes O(√n) time per number and O(1) extra space. If many queries are made for all values up to N, the Sieve of Eratosthenes preprocesses in O(N log log N) time and uses O(N) space, then answers each primality query in O(1).

Q6 – Loops That Must Terminate: Loan Repayment Schedule (8 Marks)

Each month, interest is added to the outstanding balance and a fixed repayment is subtracted. A do-while loop or while loop can print the schedule; a while loop is suitable because the balance can be checked before each month. The final payment must be capped at the amount due so that the balance does not become negative.

Correct termination and safety rules

Use a small tolerance for a floating-point balance rather than testing `balance != 0`. If the repayment is less than or equal to the current monthly interest, the principal will not decrease; with a fixed payment, the loan cannot be fully repaid. A safety limit such as 360 months prevents an accidental non-terminating run. The sample values in the question are Rs. 10,000 principal, 2% monthly interest and Rs. 2,000 fixed repayment.

C program

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    double balance = 10000.0;
    const double rate = 0.02;
    const double repayment = 2000.0;
    double total_interest = 0.0;
    int month = 0;
    const int LIMIT = 360;

    if (repayment <= balance * rate) {
        printf("Repayment is not greater than monthly interest.\\n");
        return 1;
    }

    printf("Month  Opening      Interest   Payment    Closing\\n");

    while (balance > 0.005 && month < LIMIT) {
        double opening = balance;
        double interest = opening * rate;
        double due = opening + interest;
        double payment = (repayment < due) ? repayment : due;

        balance = due - payment;
        if (balance < 0.005) balance = 0.0;

        total_interest += interest;
        month++;

        printf("%2d     %8.2f    %7.2f   %8.2f   %8.2f\\n",
               month, opening, interest, payment, balance);
    }

    if (balance <= 0.005)
        printf("Paid in %d months. Total interest = Rs. %.2f\\n",
               month, total_interest);
    else
        printf("Safety limit reached; loan is not fully repaid.\\n");

    return 0;
}
```

Hand-verified schedule

Month	Opening balance (Rs.)	Interest (Rs.)	Payment (Rs.)	Closing balance (Rs.)
1	10,000.00	200.00	2,000.00	8,200.00
2	8,200.00	164.00	2,000.00	6,364.00
3	6,364.00	127.28	2,000.00	4,491.28
4	4,491.28	89.83	2,000.00	2,581.11
5	2,581.11	51.62	2,000.00	632.73
6	632.73	12.65	645.38	0.00

The loan is cleared in 6 months. Total interest paid is Rs. 645.38. In month 6, the payment is Rs. 645.38 rather than the full Rs. 2,000 because only Rs. 645.38 is due.

Growth and alternative formula

As the repayment approaches the monthly interest, the principal reduction becomes smaller and the number of iterations increases sharply. At or below the interest amount, the process does not converge to a fully repaid loan under this fixed-payment model. A direct loan formula can estimate the number of periods in O(1), but it does not produce each monthly row and may be affected by floating-point precision and assumptions about payment timing. The simulation takes O(m) time for m months and O(1) extra space.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected program: Q5 prime-number checker. The original trial-division version checks every possible divisor from 2 to n−1. Its work grows linearly with n for prime inputs, which is unsuitable when many ticket numbers up to 1,000,003 must be checked.

A. Diagnose the bottleneck (2 Marks)

The dominant work is the repeated modulus operation `n % i`. In the original loop, a prime number performs n−2 divisibility checks because no early divisor is found. For n=1,000,003, that is 1,000,001 checks. The same work is repeated for every ticket, even when many values fall within a known maximum range.

B. Define the design objective (2 Marks)

Reduce repeated primality work for a batch of ticket queries up to a known maximum N. The measurable target is to preprocess the prime status once and answer each subsequent query in constant time, while preserving correct answers for all integers in the supported range.

C. Revised solution: Sieve of Eratosthenes (4 Marks)

Create a Boolean array is_prime[0..N], initially true. Mark 0 and 1 false. For each p from 2 while p ≤ N/p, if p is still prime, mark its multiples starting at p×p as composite. A query then reads is_prime[x]. This uses O(N log log N) preprocessing time, O(N) space and O(1) time per query. Use a sufficiently large integer type for p*p or use p <= N/p to avoid overflow.

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    const int N = 1000003;
    unsigned char *prime = malloc((N + 1) * sizeof(*prime));
    int p, x;

    if (prime == NULL) {
        printf("Memory allocation failed\\n");
        return 1;
    }

    for (x = 0; x <= N; x++) prime[x] = 1;
    prime[0] = prime[1] = 0;

    for (p = 2; p <= N / p; p++) {
        if (prime[p]) {
            int multiple;
            for (multiple = p * p; multiple <= N; multiple += p)
                prime[multiple] = 0;
        }
    }

    /* Example ticket queries */
    {
        int tickets[] = {97, 100, 7919, 1000003};
        int k;
        for (k = 0; k < 4; k++) {
            x = tickets[k];
            printf("%d: %s\\n", x,
                   prime[x] ? "prime" : "not prime");
        }
    }

    free(prime);
    return 0;
}
```

D. Prediction before implementation (1 Mark)

For a batch of many queries up to N=1,000,003, the sieve should perform one preprocessing pass and then one array lookup per ticket. It should be substantially faster than repeating up to roughly 1,000 trial divisions for each large prime, at the cost of approximately 1 MB of byte-array storage.

E. Validation and comparison (2 Marks)

Ticket	Trial division result	Sieve result	Validation
97	Prime	Prime	Match
100	Not prime	Not prime	Match

Ticket	Trial division result	Sieve result	Validation
7919	Prime	Prime	Match
1000003	Prime	Prime	Match

The supplied trial-division counts for the improved square-root loop are 8, 1, 87 and 999 respectively. The sieve returns the same classifications, but the number of checks is not directly comparable to a single-query division count because it performs shared preprocessing for the entire range.

F. Defend the final design (2 Marks)

The sieve is preferable when many ticket numbers within a known bounded range are checked, because the preprocessing cost is shared and each query is O(1). For only one or two isolated large numbers, square-root trial division may be preferable because it needs O(1) extra memory and avoids allocating the sieve. The redesign therefore improves batch throughput, but trades additional memory and preprocessing for that benefit.

Final Summary and Validation Checklist

The assignment identifies and corrects the key C issues: expression precedence and integer arithmetic, undefined behaviour from modifying a variable twice without sequencing, short-circuit logical evaluation, bit-mask precedence and signedness, switch fall-through and range validation, inefficient divisor loops, and loan-loop termination and floating-point comparison.

Validation checklist

• Compile each program using a C compiler with warnings enabled, for example: `gcc -std=c11 -Wall -Wextra program.c -o program`.
• Run the stated sample inputs and compare the output with the expected values in this report.
• For Q4, test all grade boundaries and invalid values.
• For Q5, test n values below 2, small primes, composites and the maximum ticket number.
• For Q6, test repayment greater than, equal to and less than monthly interest, and verify the safety limit.
• Keep the GitHub repository updated with the final source files.

Source and scope note

This report is based on the supplied screenshots. The exact scholarship income thresholds in Q4 and the complete detailed loan input format in Q6 are not visible in the screenshots; no missing scholarship rules have been invented. The Q4 scholarship structure is therefore explained, while the grade conversion is fully implemented. The Q6 program uses the example values explicitly shown in the question.

End of C02 Assignment